

GROUP
TWO

A Guess Number Game



A command-line number guessing game in Python

The problem

Algorithm

The functions

game()

The problem

The computer picks a secret number. The player guesses it, and after each guess the program says whether it is too low, too high or correct.

Too low

Too high

Correct!

Built in four levels, each adding more:

LEVEL 1

Basic

One random number, one guess, compare and show the result.

LEVEL 2

Intermediate

Keep guessing until correct and count the attempts.

LEVEL 3

Problem solving

Input validation, difficulty levels with different ranges, and a maximum number of attempts.

LEVEL 4

Challenge

Hints, scoring, star ratings, multiplayer, multiple rounds and a saved leaderboard.

The general algorithm



Imports: random (secret number), json (leaderboard file), os (screen and file checks), getpass (hidden guesses).

The helpers do one job each. `game()` is the controller that calls them in order.

`generate_secret_number()`

Picks the random number

`compare_guess()`

Too high, too low or correct

`choose_difficulty()`

Returns range and attempt limit

`get_valid_guess()`

Reads and validates a guess

`calculate_score()`

Fewer attempts, higher score

`get_rating()`

Turns a score into stars

`choose_game_mode()`

Single player or multiplayer

`give_hint()`

Even/odd or half-range hint

`display_leaderboard()`

Ranks best scores

Imported modules and what they are for

A module is ready-made Python code we import instead of writing it ourselves.

random

```
import random
```

Generates the secret number

`random.randint(low, high)` returns a whole number between low and high, both included.

USED IN
`generate_secret_number()`

json

```
import json
```

Saves and loads the leaderboard

`json.dump()` writes the Python dictionary to `leaderboard.json`; `json.load()` reads it back.

USED IN
`game()`

os

```
import os
```

Talks to the operating system

`os.path.exists()` checks the file;
`os.name` picks cls or clear;
`os.system()` clears the screen.

USED IN
`game()`

getpass

```
import getpass
```

Hides the typed guess

`getpass.getpass()` reads input without showing it, so the other player can't see the guess.

USED IN
`get_valid_guess()` in `multiplayer`

Function 1 of 9

`generate_secret_number(low=1, high=100)`

PARAMETERS

low and high set the range. Both have defaults (1 and 100), so the caller can leave them out.

LOGIC

`random.randint(low, high)` picks an integer between the two, including both ends.

RETURNS

The secret number (an integer).

CODE

```
def generate_secret_number(low=1, high=100):  
    rand_num = random.randint(low, high)  
    return rand_num  
  
# generate_secret_number()  
#   -> number from 1 to 100  
# generate_secret_number(1, 500)  
#   -> number from 1 to 500  
  
secret_number = generate_secret_number(1, 50)
```

Function 2 of 9

`compare_guess(guess, secret_number)`

PARAMETERS

guess is the player's number; secret_number is the target.

LOGIC

Compares the two with if / elif / else: greater, smaller or equal.

RETURNS

"Too high! Try a lower number.", "Too low! Try a higher number."
or "Correct".

CODE

```
def compare_guess(guess, secret_number):  
    if guess > secret_number:  
        return "Too high! Try a lower number."  
    elif guess < secret_number:  
        return "Too low! Try a higher number."  
    else:  
        return "Correct"  
  
# It returns text instead of printing it,  
# so logic stays apart from display.
```

Function 3 of 9

choose_difficulty()

PARAMETERS

None. It asks the player to pick Easy, Medium or Hard.

LOGIC

Maps the choice to a range and an attempt limit, as in the table on the right.

RETURNS

Three values as a tuple: low, high, max_attempt.

CODE

```
# Easy    1-50    5 attempts
# Medium  1-100   7 attempts
# Hard    1-500   8 attempts

# e.g. Easy:
return 1, 50, 5

# the caller unpacks the tuple:
low, high, max_attempt = choose_difficulty()
```

Function 4 of 9

`get_valid_guess(low, high, mode)`

PARAMETERS

low and high are the valid range. mode picks input() (single) or getpass (multiplayer, hidden).

LOGIC

while True: read a guess, convert it with int(), check low <= guess <= high. Bad input is rejected.

RETURNS

A valid integer guess. Invalid guesses never cost an attempt.

CODE

```
def get_valid_guess(low, high, mode):
    while True:
        # input() or getpass.getpass()
        try:
            guess = int(guess)
            if low <= guess <= high:
                return guess
        except ValueError:
            print("Invalid input.")

# 'abc' -> ValueError -> asks again
# 700 in 1-500 -> rejected
```

Function 5 of 9

calculate_score(attempts, max_attempt)

PARAMETERS

attempts is how many guesses were used; max_attempt is the limit for the difficulty.

LOGIC

Fewer attempts means a higher score: $((\text{max} - \text{attempts} + 1) / \text{max}) \times 100$, rounded.

RETURNS

The score as a rounded number.

CODE

```
def calculate_score(attempts, max_attempt):  
    score = ((max_attempt - attempts + 1)  
             / max_attempt) * 100  
    return round(score)  
  
# Medium (max 7):  
# 1 attempt -> 100  
# 3 attempts -> 71  
# 7 attempts -> 14
```

Function 6 of 9

get_rating(score)

PARAMETERS

score is the number from calculate_score().

LOGIC

A chain of if / elif checks turns the score into stars, from one star up to five.

RETURNS

A star string. Scoring and presentation stay separate.

CODE

```
def get_rating(score):  
    if score <= 20:  
        return "★"  
    elif score <= 40:  
        return "★★"  
    # ... and so on,  
    # up to five stars
```

Function 7 of 9

choose_game_mode()

PARAMETERS

None. It shows 1. Single Player and 2. Multiplayer.

LOGIC

Repeats until the answer is in the tuple ("1", "2").

RETURNS

The valid choice, "1" or "2", as a string.

CODE

```
def choose_game_mode():  
    while True:  
        print("1. Single Player")  
        print("2. Multiplayer")  
        mode = input("Enter your choice: ")  
        if mode in ("1", "2"):  
            return mode  
        print("Invalid game mode.")
```

Function 8 of 9

```
give_hint(secret_number, low, high, wrong_guess)
```

PARAMETERS

The secret number, the range, and wrong_guess: how many full rounds of wrong guesses so far.

LOGIC

At 3 it says even or odd using % 2. After that it uses the midpoint to say which half the number is in.

RETURNS

A hint string.

CODE

```
if wrong_guess == 3:
    if secret_number % 2 == 0:
        # hint: the number is even
    else:
        # hint: the number is odd
else:
    midpoint = (low + high) // 2
    # hint: lower half or upper half

# 8 % 2 = 0 (even)    9 % 2 = 1 (odd)
# 10 // 3 = 3
```

Function 9 of 9

`display_leaderboard(leaderboard)`

PARAMETERS

leaderboard is a dictionary of player name to best score.

LOGIC

Turns it into items, sorts by score with a lambda, highest first, and numbers the rows with enumerate().

RETURNS

Nothing. It shows the ranked list.

CODE

```
# simplified sketch
ranked = sorted(
    leaderboard.items(),
    key=lambda item: item[1],
    reverse=True,
)
for place, (name, score) in enumerate(ranked, 1):
    print(f"{place}. {name} - {score}")

# {"John": 70, "Mary": 90}
# 1. Mary - 90
# 2. John - 70
```

game(): the controller

1 Setup

Clear screen, load or create the leaderboard, choose mode, enter unique names.

2 Start a round

Choose difficulty, generate the secret number, reset attempts and counters.

3 Turn loop

Get a valid guess, count it, compare. Correct ends the round; wrong updates counters, hints and the turn.

4 Round end

Winner: score, rating, save best. No winner: game over, nothing recorded.

5 Menu

Play again, view leaderboard or exit, then show the final leaderboard.

PSEUDOCODE

```
START game
  clear screen
  load or create leaderboard
  choose mode, get player names
  WHILE playing
    choose difficulty
    generate secret number
    set attempts, counters, winner = None
    WHILE no winner AND attempts left
      get valid guess, +1 attempt, compare
      IF correct: set winner, stop
      ELSE: count wrong, hint every 3rd
      move to next player
    IF winner: score, rating, save best
    ELSE: game over
    menu: again / leaderboard / exit
  show final leaderboard
END game
```

game() code: setup

`os.name`

"nt" means Windows, so cls; otherwise clear.

`os.path.exists + with open`

Create an empty JSON leaderboard, or load the saved one. `with` closes the file for us.

`player_names list`

`.strip()` trims spaces; names must be non-empty and unique.

`json.dump / json.load`

Save and restore the leaderboard between runs.

CODE

```
if os.name == "nt":
    os.system("cls")
else:
    os.system("clear")
leaderboard_file = "leaderboard.json"
if not os.path.exists(leaderboard_file):
    leaderboard = {}
    with open(leaderboard_file, "w") as file:
        json.dump(leaderboard, file, indent=4)
else:
    with open(leaderboard_file, "r") as file:
        leaderboard = json.load(file)
player_names = []
for player_number in range(1, number_of_players + 1):
    while True:
        name = input(...).strip()
        if not name:
            print("Name cannot be empty.")
        elif name in player_names:
            print("That name is already taken.")
        else:
            player_names.append(name)
            break
```

game() code: round and turn loop

attempts = [0] * players

One counter per player: 3 players gives [0, 0, 0].

winner = None

Nobody has won yet. On a win it holds that player's index.

current_player = 0

Zero-based index: 0 is the first player.

Loop condition

Runs while there is no winner and this player still has attempts.

CODE

```
low, high, max_attempt = choose_difficulty()
secret_number = generate_secret_number(low, high)
attempts = [0] * number_of_players
wrong_guesses = 0
round_wrong_guesses = 0
winner = None
current_player = 0

while (
    winner is None
    and attempts[current_player] < max_attempt
):
    guess = get_valid_guess(low, high, mode)
    attempts[current_player] += 1
    result = compare_guess(guess, secret_number)
    print(result)

    if result == "Correct":
        winner = current_player
        break
```

game() code: hints and turns

round_wrong_guesses

Counts wrong guesses in the current cycle.

wrong_guesses

Counts full cycles where every player missed.

% 3 == 0

Every third full wrong cycle triggers give_hint().

(current + 1) % players

Wraps back to 0 after the last player: 0, 1, 0, 1 ...

CODE

```
# inside the turn loop, after a wrong guess
round_wrong_guesses += 1

if round_wrong_guesses == number_of_players:
    wrong_guesses += 1
    round_wrong_guesses = 0

    if wrong_guesses % 3 == 0:
        print(give_hint(secret_number, low, high,
                        wrong_guesses))

current_player = (
    current_player + 1
) % number_of_players
```

game() code: results and menu

Winner found

Score and rating are calculated for the winner.

Best score only

A lower new score never replaces a higher saved one.

Save

The dictionary is written back to leaderboard.json.

Menu

1 play again, 2 leaderboard, 3 exit (playing = False), then the final leaderboard.

CODE

```
# simplified sketch
if winner is not None:
    name = player_names[winner]
    score = calculate_score(attempts[winner],
                           max_attempt)
    print(get_rating(score))

    if score > leaderboard.get(name, 0):
        leaderboard[name] = score

    with open(leaderboard_file, "w") as file:
        json.dump(leaderboard, file, indent=4)
else:
    print("Game over!")
    print(f"The number was {secret_number}")

# after the round: show the menu
# 1 -> new round    2 -> display_leaderboard()
# 3 -> playing = False
```



Thank you

Questions?



GROUP TWO • A Guess Number Game